AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_vhdl2_ingenjorsjobb_b.pdf Version : V1.0	

TEIS ECS - Embedded Computer System – PROTOTYP AV AVSTÅNDSMÄTARE MED EN ULTRALJUDSENSOR

Darwin Rengifo

2019-18-11

Sammanfattning:
Den rapport beskriver en förstudie och en planering för projektet av avståndsmätare med ultraljudsensor med resultat visualisering på sjusegmentsdisplayerna som finns på utvecklingskortet och VGA-skärmen som ska det visar färger röd, grön, blå och vit för olika avstånd. Denna rapport innehålla aktivitets- och tidplan, projektet kostnad, kravspecifikations, projektet arkitektur och komponenterna beskrivning som ska utvecklas.

Innehåll

1. INLEDNING 4
2. KRAVSPECIFIKATION FRÅN KUND 4
3. VHDL-PROJEKT FRÅN TIDIGARE ÅR 7
4. PROJEKPLAN 7
5. KRAVSPECIFIKATION 8
6. ARKITEKTUR FÖR INGÅENDE KOMPONENTER 9
   - 6.1 KOMPONENTERNA TILL PROJEKTET 9
   - 6.2 Vhdl2_ingenjorsjobb 10
   - 6.3 Trigger_cont 10
   - 6.4 Echo_cont 10
   - 6.5 Distans_raknarem 10
   - 6.6 Bcd_converter 10
   - 6.7 Color_delar 10
   - 6.8 Sju_seg_displayer 10
   - 6.9 Vhdl2_uppgift_4 10
   - 6.10 Display_rgb 10
   - 6.11 Vga_synkronisering 11
7. KONSTRUKTIONS BESKRIVNING 11
   - 7.1 SYSTEM ARKITEKTUR 11
   - 7.2 DELSYSTEM 12
   - 7.2.1 TOPPNIVÅN 12
   - 7.2.1.1 SYSTEM ARKITEKTUR 12
   - 7.2.2 TRIGGER - KOMPONENT 13
   - 7.2.2.1 SYSTEM ARKITEKTUR 13
   - 7.2.3 ECHO - KOMPONENT 14
   - 7.2.3.1 SYSTEM ARKITEKTUR 14
   - 7.2.4 BERÄKNAR AVSTÅND - KOMPONENT 15
   - 7.2.4.1 SYSTEM ARKITEKTUR 15
   - 7.2.5 VÄLJER FÄRGER - KOMPONENT 16
   - 7.2.5.1 SYSTEM ARKITEKTUR 16

7.2.6 VGA KONTROLLER - KOMPONENT 16
7.2.6.1 SYSTEM ARKITEKTUR 17
7.2.6.2 COUNTER – KOMPONENT 18
7.2.6.2.1 SYSTEM OCH ARKITEKTUR 18
7.2.6.3 DISPLAY RGB – KOMPONENT 19
7.2.6.3.1 SYSTEM OCH ARKITEKTUR 19
7.2.7 BCD KONVERTER - KOMPONENT 20
7.2.7.1 SYSTEM ARKITEKTUR 20
7.2.8 SJU_SEG_DISPLAYER - KOMPONENT 20
7.2.8.1 SYSTEM ARKITEKTUR 21
7.3 ULTRALJUDSENSOR HC-SR05 21
7.3.1 I/O PINNAR 22
7.4 SDC FILEN 22
8. SETTINGS OCH ASSIGNMENTS FÖRÄNDRING 23
8.1 SETTINGS 23
8.2 ASSIGNMENTS 23
9. VERIFIERING 23
9.1 TESTBÄNKEN 23
9.2 RESULTATEN FRÅN SIMULERINGEN 24
10. ANALYS 25
10.1 FELMEDDELANDEN 25
10.2 FREKVENS OCH STORLEK FÖRE OPTIMERING 25
10.2.1 MAX FREKVENS FÖR HELA KONSTRUKTIONEN 25
10.2.2 STORLEK FÖR HELA KONSTRUKTIONEN OCH DEL SYSTEM 26
10.3 EFFEKTFÖRBRUKNING UPPSKATTNING 27
10.4 TIDSVALIDERING 27
11. OPTIMERING KONSTRUKTIONEN 27
12. VALIDERING KONSTRUKTIONEN 28
12.1 TESTPROTOKOLL FÖR VALIDERING 28
12.2 VALIDERING FILM 28
13. REFERENSER 29

1. INLEDNING

Denna rapport beskriver som bygger en avståndsmätare med ultraljudsensor och alla komponenterna som kommer att bygga för att se resultat av visualisering på sjusegmentsdisplayerna som finns på utvecklingskortet och VGA-skärmen som ska det visar färger röd, grön, blå och vit för olika avstånd. Det kommer att analyserats systemet genom simulering och verifiering i ModelSim och därefter validerats på ett lämpligt FPGA-kort.

2. KRAVSPECIFIKATION FRÅN KUND

Tabell 1. Kravspecifikation från kund

Krav id	Beskrivning	Utfört Ja/nej
	Förstudie	
0	Beskriv två VHDL-projekt från tidigare år som helst har någon likhet med det som ska utföras i detta ingenjörsjobb.	
1a	Aktivitets- och tidplan för projektet. Följ upp tidplanen och redovisa resultatet i slutrapporten. Lön är 500 SEK/tim.	
1b	En separat kravspecifikationstabell. Beskriv det som ska realiseras med en inledning och en kravspecifikation. Kravspecifikationen får förändras under projektets gång.	
1c	Gör en första arkitektur som visar alla ingående komponenter (kan senare modifieras). Beskriv också de komponenter som ska utvecklas, återanvändas från kursen och andras komponenter med referenser.	
	Förstudien avslutad **----------** **Start av slutleverans**	
	Konstruktions krav	
3a	Efter kompilering av projektet, ska det inte finnas några ”critical warnings” kvar. Om det finns varningar kvar ska dessa beskrivas.	
3b	Regler och riktlinjer för TEIS ska följas (de som är markerade med ”**M**”.	
	Rapportkrav	

4	Utgå gärna från teknisk_rapport_mall, som ligger på Itslearning under kursen Konstruktionsmetodik och teknisk dokumentation. Kunden kräver en standardiserad teknisk rapport med filnamn ”fornamn_efternamn_VHDL2_ingenjorsjobb”. Följande ska finnas med i slutrapporten: • Kravspecifikation, eget kapitel. Återanvänd från förstudier/startrapport med modifieringar. • Beskrivning av två äldre projekt (som liknar det valda projektet) med några kommentarer om förbättringar för att förstå konstruktionen, eget kapitel (krav 0). Återanvänd från startrapport. • Projektplan, aktiviteter och en budgeterad kostnad (500 SEK/tim), eget kapitel (krav 1a). Återanvänd från startrapport men lägg till uppföljning. • Testprotokoll för verifiering/validering, grundat på kravspecifikation 2 vilken togs fram av ingenjören i början. Protokollet behöver inte vara komplett, det räcker att visa hur det ska utföras och det behöver bara visas på systemnivå (inte på subnivåer), eget kapitel. • Konstruktions beskrivning, eget kapitel o System arkitektur (”top-down” beskrivning) o Delsystems beskrivning i underkapitel o I/O pinnar, Metastabilitetsproblem och sdc (gör en enkel sdc fil) • Verktygs- ”settings och assignments”, eget kapitel o Dokumentera alla förändrade kompileringsinställningar för ”settings och assignments” • Verifiering, eget kapitel o Teststimuli (som bygger på testprotokollet för konstruktionen) o Figurer med beskrivning på några viktiga resultat från simuleringen med ModelSim • Analys, eget kapitel	

	○ Felmeddelanden ”Warning” från kompileringen som finns kvar, bör kommenteras. ”Critical Warning” ska inte finnas kvar efter kompileringen ○ Max frekvens och storlek från Quartus-rapporterna ✦ Beskrivning av storlek för hela konstruktionen och varje delsystem (icke optimerad konstruktion) ○ Kommentera ”Design Assistant förslag” **(frivillig)** ○ Effektförbrukningsuppskattning ○ Tidsvalidering, SDC resultat med slack • Optimering, eget kapitel ○ Optimera konstruktionen ur storlek, visa före och efter optimeringen ○ Beskriv hur du har optimerat konstruktionen med hjälp av verktygen och eventuellt förändring i VHDLkoden • Validering, eget kapitel ○ Visa med resultat från testprotokollet (validering) ○ Kombineras med en film (max 15 minuter) som beskriver för en utomstående vad som genomförts (visa gärna en systemarkitektur) och demonstrera att det fungerar (en länk till YouTube). Träna gärna på presentationsteknik, tänk på att det är en gymnasieelev som tittar på filmen (kunden granskar inte detta moment, mer än ser att det fungerar). • Bilaga: Lista och kort beskrivning på IP-komponenter som kan återanvändas från projektet/kursen	
Leveranskrav av slutleveransen		
5	Slutleveransen ska innehålla två mappar med följande dokument: • **<u>Mappen Dokumentation</u>** ○ Konstruktionsrapporten (se rapportkrav) ○ Eventuellt bilagor som kod, kretsschema och datablad • **<u>Mappen Konstruktionsunderlag</u>** ○ Arkiverat Quartus projekt, ○ Större komponenter, t.ex. VGA -rapporten läggs också med. Dokumentation och kod från föregående jobb (återanvänd). • **<u>Mappen Konfigureringsfil</u>** ○ configuration.sof • **Mappen diverse** Leveransen ska ske till plattformen Itslearning. Den slutgiltiga leveransen är hela jobbet och namnet på filen ska vara ”fornamn_efternamn_VHDL2_ingenjorsjobb_b.zip”.	

3. VHDL-PROJEKT FRÅN TIDIGARE ÅR

Pulsgivarens rotation (se referenser)
Det projektet visar roteranden pulsgivare användningen för att ökar eller minskar numren på sjusegmentdisplayerna.
Pulsgivarens rotation omvandlas till pulser, som behandlas i styrsystemet. Pulsgivaren ger två pulståg som är förskjutna i förhållande till varandra. Genom att jämföra pulstågen kan mätverket avgöra om matningen går framåt eller bakåt.
Det styrsystemet har tre komponenter som är metastabilitet för att säkerställa tillförlitlighet och synkroniserar asynkron signaler med klockan. Den komponenten innehålla systemklock, pulståg(GPIO), key(Key_0, Key_1) och reset. En tillståndsmaskin för att exekverar två test case (aktiveras via tryckknapparna: Key_0, Key_1) som kommer att börjar en räkning för att ökar eller minskar numren för att visa på sjusegmentdisplayerna. Sjusegmentdisplayerna för at visa test case resultaten.

Motordrivare (se referenser)
Projektet visar rotation styr av en spegel igenom av två motorer som är kontrollerar med en motordrivare för varje motor, en brytare(SW0) för att byta mellan motorerna. KEY_0 och KEY_1 för att roterar spegel på horisontell eller vertikal axlar. En VGA-skärm för att visa i siffror rotationen resultatet. Projektet innehålla fyra komponenter: En topp nivå komponenten som instansierar motordrivare komponenten, komponenten som styr utgången till display VGA och en PLL komponenten som ändrar systemklock frekvensen. Motordrivare komponenten får alla ingången signaler för att kan styr rotation av motorerna denna rotation går till en klock frekvens av 10Mhz. Den komponenten PLL_25Mhz får det 50Mhz systemklock för att ändra frekvens till 25Mhz för att kan synkroniserar utgångarna till VGA-skarmen. Vga_sync komponenten användas för att räknar skärm pixel för att kan aktivera horisontell och vertikal synkronisering.

4. PROJEKPLAN

Aktivitets - och tidplan för ingenjörsjobbet kan ses i Tabell 2. Totalt uppskattas ingenjörsjobbet ta 80 timmar/500kr(timme), vilket ger en lönekostnad på 40 000 SEK.

Tabell 2. Aktivitets- och tidsplan för ingenjörsjobbet

Aktivitet	Veckor				
	46	47	48	Timmar(h)	Kostnad(kr)
Kravspec./förstudie	13			13	6 500
Testprotokoll/ Drivrutiner	4	4	1	9	4 500
Arkitekturbeskrivning	5	3		8	4 000
Programmering av visuellt gränssnitt	8	12	8	28	14 000
Verifiering		2	2	4	2 000

Validering			2	2	1 000
Buggrättning		6	3	9	4 500
Slutrapport		4	3	7	10 000
Summa tid	30	31	19	80	40 000

5. KRAVSPECIFIKATION

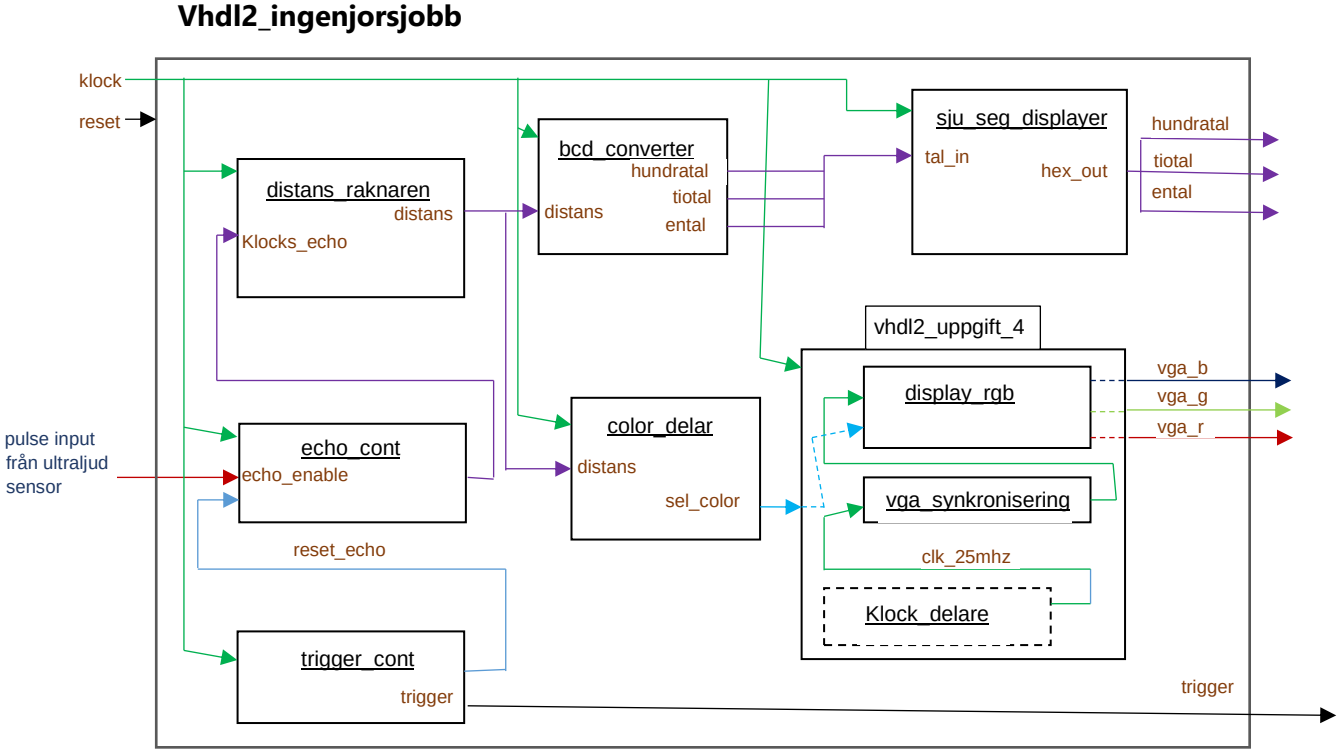
Tabell 3. Kravspecifikation från vhdl2_ingenjorsjobbet_a

Krav id	Beskrivning	Utfört Ja/nej
	Förstudie	
1a	VGA-protokollet ska följas och 50MHz klockan ska delas ned med en process y toppnivån.	Ja
1b	Konstruera en toppnivån"vhdl2_ingenjorsjobb" komponent som innehåller alla komponenterna söm göra avståndsmätare.	Ja
1c	Konstruera en komponenterna för aktiverar ultraljudsensor för att kan läsa avstånd och konstruera komponenter för att visar avstånd på sjusegmentsdisplayerna.	Ja
1d	Konstruera en VGA komponent" vhdl2_ vhdl2_uppgift_4" för att får ingången signaler, instansierar de andra komponenter och skickar utgången signaler till skärmen för att visar färger röd, grön, blå och vit för olika avstånd.	Ja
	Testprotokoll	
2	Verifiering: Beskriv test fall. Validering på FPGA kortet: Beskriv fyra testfall, ett som startas med 10cm, 20cm, 30cm och över 30cm. Det ska ritas ut på skärmen färgen röd, grön, blå och vit för olika avstånd.	Ja
	VHDL-kod	
3	Regler och riktlinjer för VHDL och C ska följas.	Ja
	Verifierings-/Valideringskrav	
4a	Verifiera med ModelSim. Fyll i testprotokollet.	Ja
4b	Validera på DE1-SOC kortet. Fyll i testprotokollet.	Ja

6. ARKITEKTUR FÖR INGÅENDE KOMPONENTER.

HC-SR05-sensorn är en modul som innehåller ett par transducer som används tillsammans för att bestämma avståndet mellan sensorn och ett objekt placerar framför den. En transducer avger ultraljud signaler och den andra tar upp studs för den vågen. Den tid som det tar ljudvågen att gå och återgå till ett objekt kan användas för att veta avståndet mellan ljudets ursprung och objektet.

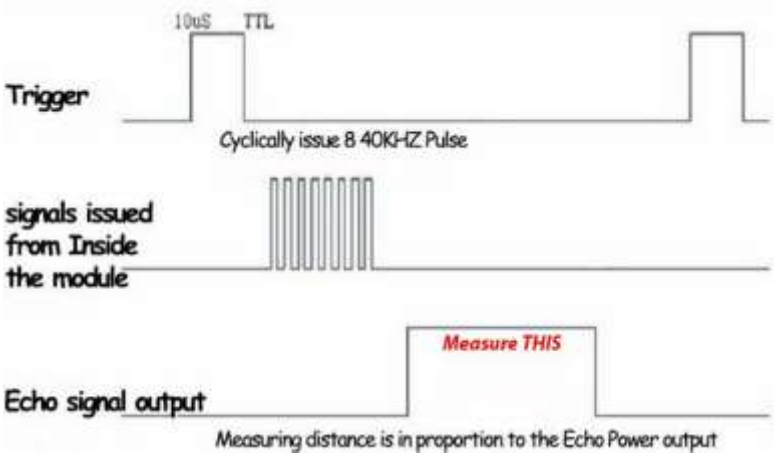
På figur 1, visar det projektet arkitektur med alla komponenter som kommer att använder på avståndsmätare projektet.



Figur 1. Toppnivån som instansierar alla komponenter för avståndsmätare

6.1 KOMPONENTERNA TILL PROJEKTET

För att konstruerar projektet måste det utveckla komponenterna för att läsa information från sensorn ”HC-SR05”, räknar avstånd, visar avståndet på sjusegmentsdisplayerna i kortet, visar färgerna på skärmen för att validerar avståndet. Figur 2 visar diagrammet för tidläsning av ultraljudsensor.



Figur 2. Ultaljudsensor timing diagram

6.2 Vhdl2_ingenjorsjobb

Top filen som kommer att instansierar alla komponenterna för avståndsmätare med ultraljudsensor HC-SR05 och sen visa på skärmen olika färger beroende på avstånd: 10cm = röd, 20cm=grön, 30cm=blå, efter 30cm=vit.

6.3 Trigger_cont

Den ”trigger” pin får en aktivering puls av 10us från mikrokontroll genom denna puls indikerar till sensorn att måste börjar med avståndsmätaren.

6.4 Echo_cont

Denna komponent kontrollerar ”echo” räknaren. Räknar hur många klockpulser det finns på en ”echo” period. Igenom ”echo” pin, detta sensorn returnerar till mikrokontroll en puls vars bredd är proportionell till tiden som tar ljudet att resa från transducer till objekt och sen återgå till sensorn.

6.5 Distans_raknarem

Efter att ”echo” blir det negativ flank, den komponent får antal klockpulser total för att räknar avstånd som finns mellan ultraljudsensor och ett objekt.

6.6 Bcd_converter

Denna komponent omvandlar avstånd binär värde till bcd värde för att visa efter det på 7-segment display. Det kommer att använder metoden shift-and-add-3.

6.7 Color_delar

Denna komponent får en signal av 3-bitar från ”distans_raknaren” komponenten för att väljer färgerna som kommer att rita på skärmen.

6.8 Sju_seg_displayer

Styrprogram för 7-segment display på FPGA kort för att visar avståndet i centimeter.

6.9 Vhdl2_uppgift_4

Topp filen som instansierar alla komponenterna för att ritas ut färgerna på skärmen.

6.10 Display_rgb

Den visar var röd, grön och blå färg ska ritas ut på skärmen enligt testfallen.

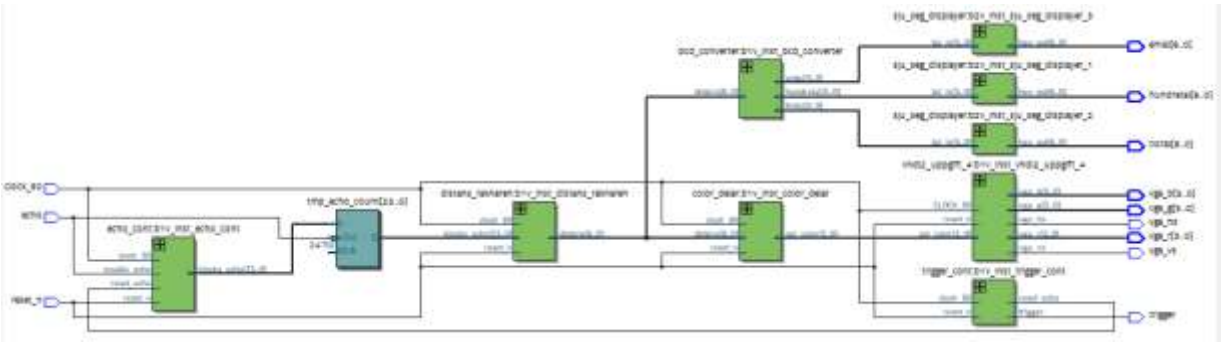
6.11 Vga_synkronisering

Räknar en skärm av 640 x 480 pixel för att aktivera horisontell, vertikal och mörk synkronisering.

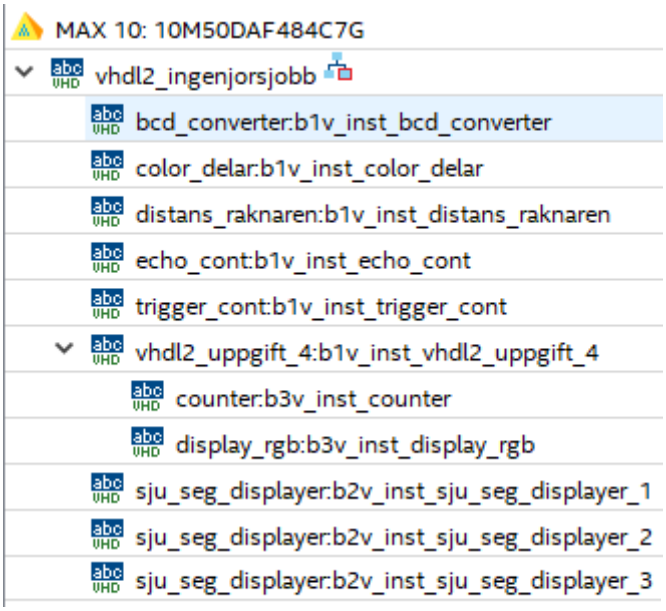
7. KONSTRUKTIONS BESKRIVNING

Programmet körs på kortet DE10-Lite är en robust hårdvara design plattform som byggde kring Altera MAX 10 FPGA med en Nios II-CPU som är anpassad för ingenjörsjobbet. DE10-Lite-kortet innehåller en 15-pin D-SUB-kontakt för VGA-utgång, VGA skärm för att visualisering resultaten, VGA-kabel för att skickar signalerna från DE10-Lite kortet till VGA skärm, en ultraljudsensor HC-SR05 för att göra avståndsmätare testen och en protoboard för att koppla in kablarna mellan DE10-Lite-kortet pinnes och ultraljudsensor HC-SR05.

7.1 SYSTEM ARKITEKTUR



Figur 3. Systemarkitekturen för TEIS ulraljudsensor



Figur 4. Komponenthierarki för ultraljudsensor

7.2 DELSYSTEM

Systemet innehålla följande komponenter:

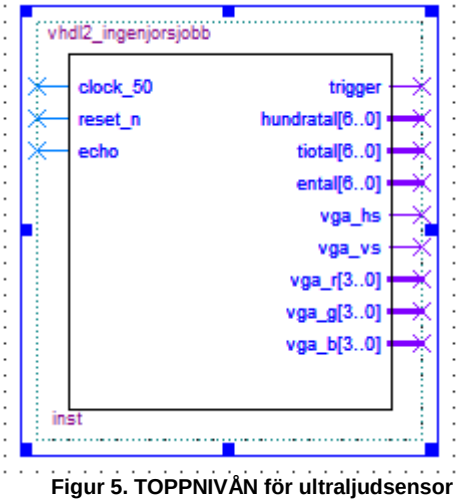
7.2.1 TOPPNIVÅN

Komponent namn: ***vhdl2_Ingenjorsjobb***

7.2.1.1 SYSTEM ARKITEKTUR

Den toppnivån användas för att instansierar de alla projektkomponenterna. Toppnivån får ingången signaler som instansierar komponenterna och skickar utgången signaler till sjusegmentdisplayerna för at visa mätning resultat även skickar utgången signaler till skärmen för att rita olika färger för olika avstånd.

TOPPNIVÅN symbol visas i Figur 5 och tabell 4 visar In- och utsignalerna.



Figur 5. TOPPNIVÅN för ultraljudsensor

Tabell 4. In- och utsignalerna till TOPP filen

Signal	**Namn**	**Riktning**	**Typ**
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Echo timer	echo	in	std_logic
Trigger-10us puls	trigger	out	std_logic
7-segmentsdisplay till hundratal	undratal	out	std_logic_vector(6 downto 0)
7-segmentsdisplay till tiotal	tiotal	out	std_logic_vector(6 downto 0)
7-segmentsdisplay till ental	ental	out	std_logic_vector(6 downto 0)
Horizontell synkronism	vga_hs	out	std_logic
Vertikal synkronism	vga_vs	out	std_logic

Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)

Ingången signaler

CLOCK_50: 50Mhz input clock.
reset_n: Reset av signalen. Active low.
echo: Pin som aktiverar sig för att börjar mäta avståndet. ”1” det börjar att räknar, ”0” slutar att räknar.

Utgången signaler

trigger: Pin som får aktiveringen puls efter 10us för att kan börjar avståndsmätare.
undratal: visa 7-segmentsdisplay till hundratal.
tiotal: visa 7-segmentsdisplay till tiotal.
entital: visa 7-segmentsdisplay till ental.
vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.
vga_vs: Vertikal synkronism. Kontrollerar början och slutet på en ram(framme) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

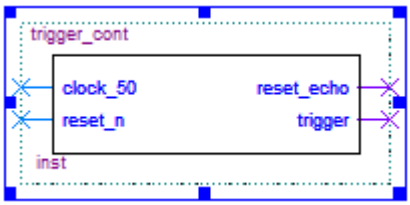
7.2.2 TRIGGER - KOMPONENT

Komponent namn: ***trigger_cont***
*Instansnamn****: b1v_inst_trigger_cont***

7.2.2.1 SYSTEM ARKITEKTUR

Pin trigger av sensorn får en puls av minst 10us från FPGA, igenom denna puls indikerar till sensorn att kan börjar avståndsmätare.

TRIGGER-komponentens symbol visas i Figur 6 och tabell 5 visar In- och utsignalerna.



Figur 6. TRIGGER 10us för ultraljudsensor

Tabell 5. In- och utsignalerna till TRIGGER filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Echo reset	reset_echo	out	std_logic
Trigger-10us puls	trigger	out	std_logic

Ingången signaler

CLOCK_50: 50Mhz input clock.
reset_n: reset av signalen. Active low.

Utgången signaler

reset_echo: efter 10us det blir reset='0' för att initierar echo räknaren.
trigger: Pin som får aktiveringen puls efter 10us för att kan börjar avståndsmätare.

7.2.3 ECHO - KOMPONENT

Komponent namn: ***echo_cont***
*Instansnamn****: b1v_inst_echo_cont***

7.2.3.1 SYSTEM ARKITEKTUR

Igenom echo pin, den sensorn återvänder till FPGA en puls vars bredd är proportionell till tiden som ljud tar att resa från sensorn till hinder och sen återgår till sensorn igen.

ECHO-komponentens symbol visas i Figur 7 och tabell 6 visar In- och utsignalerna.



Figur 7. ECHO räknaren för ultraljudsensor

Tabell 6. In- och utsignalerna till ECHO filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Echo reset	reset_echo	in	std_logic
Echo enable	enable_echo	in	std_logic

Nro. klocks echo	klocks_echo	out	std_logic_vector(23 downto 0)

Ingången signaler
CLOCK_50: 50Mhz input clock.
reset_n: reset av signalen. Active low.
reset_echo: efter 10us det blir reset='0' för att initierar echo räknaren.
Enable_echo: med stigande flank '1' börjar att räknar antal klocks och med fallande flank '0' slutar att räknar antal klocks

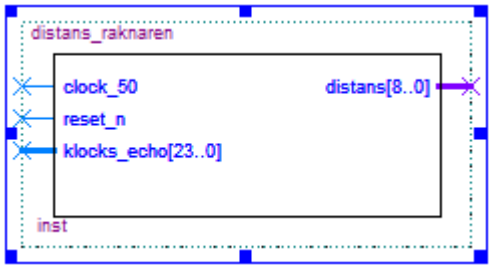
Utgången signaler
klocks_echo: ökar puls-klockor när enable_echo='1'.

7.2.4 BERÄKNAR AVSTÅND - KOMPONENT

Komponent namn: ***distans_raknaren***
*Instansnamn****: b1v_inst_distans_raknaren***

7.2.4.1 SYSTEM ARKITEKTUR

Denna komponent det kommer att få echo total bredd och igenom en beräkning det kommer att få avstånd i centimeter.
Den komponent räknar avstånd som finns mellan ulltraljudsensor och ett objekt.
DISTANS RÄKNAREN-komponentens symbol visas i Figur 8 och tabell 7 visar In- och utsignalerna.



Figur 8. AVSTÅND beräkning för ultraljudsensor

Tabell 7. In- och utsignalerna till DISTANS_RAKNAREN filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Nro. klocks echo	klocks_echo	in	std_logic_vector(23 downto 0)
Avstånd	distans	out	std_logic_vector(8 downto 0)

Ingången signaler
CLOCK_50: 50Mhz input clock.
reset_n: reset av signalen. Active low.
klocks_echo: summa total av puls-klockor när enable_echo='0'.

Utgången signaler

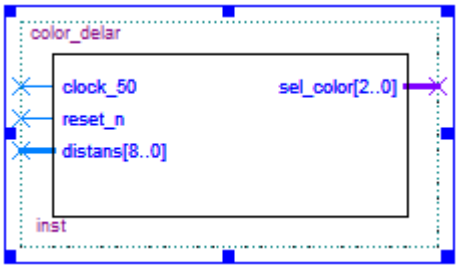
distans: avstånd i centimeter.

7.2.5 VÄLJER FÄRGER - KOMPONENT

Komponent namn: ***color_delar***
*Instansnamn****: b1v_inst_color_delar***

7.2.5.1 SYSTEM ARKITEKTUR

Denna komponent får en signal av 3-bitar från distans_raknaren för att väljer färger som kommer att visa på skärmen.
VÄLJER FÄRGER-komponentens symbol visas i Figur 9 och tabell 8 visar In- och utsignalerna.



Figur 9. FÄRGER för att visa på vga-skärmen. Ultraljudsensor

Tabell 8. In- och utsignalerna till DISTANS_RAKNAREN filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Avstånd	distans	in	std_logic_vector(8 downto 0)
Välja färgen	sel_color	out	std_logic_vector(2 downto 0)

Ingången signaler
CLOCK_50: 50Mhz input clock.
reset_n: reset av signalen. Active low.
distans: avstånd i centimeter.

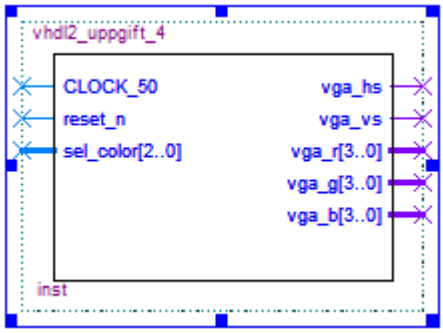
Utgången signaler
sel_color: väljer färgen.3-bitar, en bit för varje färg.

7.2.6 VGA KONTROLLER - KOMPONENT

Komponent namn: ***vhdl2_uppgift_4***
*Instansnamn****: b1v_inst_vhdl2_uppgift_4***

7.2.6.1 SYSTEM ARKITEKTUR

Denna komponent instancierar komponenterna för att visa färgerna på skärmen.
VGA KONTROLLER-komponentens symbol visas i Figur 10 och tabell 9 visar In- och utsignalerna.



Figur 10. VGA kontroller för ultraljudsensor

Tabell 9. In- och utsignalerna till VGA kontroller filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Välja färgen	sel_color	in	std_logic_vector(2 downto 0)
Horizontell synkronism	vga_hs	out	std_logic
Vertikal synkronism	vga_vs	out	std_logic
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)

Ingången signaler

CLOCK_50: 50Mhz input clock.
reset_n: reset av signalen. Active low.
sel_color: väljer färgen. 3-bitar, en bit för varje färg.

Utgången signaler

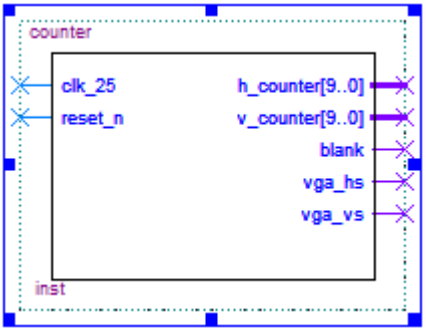
vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.
vga_vs: Vertikal synkronism. Kontrollerar början och slutet på en ram(framme) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

7.2.6.2 COUNTER – KOMPONENT

Komponent namn: ***counter***
*Instansnamn****: b3v_inst_counter***

7.2.6.2.1 SYSTEM OCH ARKITEKTUR

Den komponent användas för att räknar en skärm av 640 x 480 pixel för att aktivera horisontell, vertikal och mörk synkronisering.
COUNTER-komponentens symbol visas i Figur 11 och tabell 10 visar In- och utsignalerna.



Figur 11. COUNTER för VGA-kontroller

Tabell 10. In- och utsignalerna till COUNTER filen

Signal	**Namn**	**Riktning**	**Typ**
Klocksignal 25 MHz	clk_25	in	std_logic
Switch reset	reset_n	in	std_logic
Horizontell counter	h_counter	out	std_logic_vector(9 downto 0)
Vertikal counter	v_counter	out	std_logic_vector(9 downto 0)
Mörk skärm	blank	out	std_logic
Horizontell syn	vga_hs	out	std_logic
Vertikal sync	vga_vs	out	std_logic

Ingången signaler

clk_25: 25Mhz input clock för VGA synkpulser och räknare.
reset_n: Reset av signalen. Active low.

Utgången signaler

h_counter: Det kommer att räkna från 0 till 799, även om den synliga bilden endast är 640 pixlar bred.
v_counter: Det kommer att räkna från 0 till 525, även om den synliga bilden endast är 480 pixlar hög.

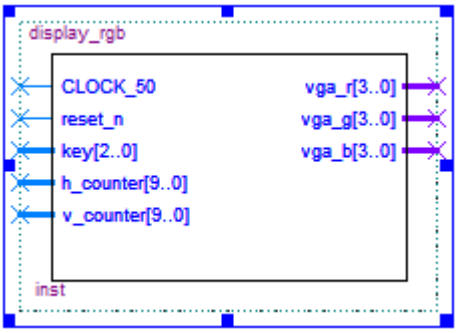
blank: Det kommer att aktiverar(”0”) när Horisontellräknare(h_counter) är mellan (640 till 799).
vga_hs: Det kommer att aktiverar(”0”) när h_counter är mellan (660 till 755)
vga_vs: Det kommer att aktiverar(”0”) när "v_counter= 494".

7.2.6.3 DISPLAY RGB – KOMPONENT

Komponent namn: ***display_rgb***
*Instansnamn****: b3v_inst_display_rgb***

7.2.6.3.1 SYSTEM OCH ARKITEKTUR

Den komponent användas för att visar var röd, grön och blå färg ska ritas ut på skärmen enligt testfallen.
DISPLAY_RGB-komponentens symbol visas i Figur 12 och tabell 11 visar In- och utsignalerna.



Figur 12. DISPLAY RGB för VGA-kontroller

Tabell 11. In- och utsignalerna till DISPLAY_RGB filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Bitar(sel_color)	key	in	std_logic_vector(2 downto 0)
Horizontell counter	h_counter	in	std_logic_vector(9 downto 0)
Vertikal counter	v_counter	in	std_logic_vector(9 downto 0)
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)

Ingången signaler
CLOCK_50: 50Mhz input clock.
reset_n: Reset av signalen. Active low.
key: For att rita fyrkanter i vissa skärmen positioner.

Utgången signaler

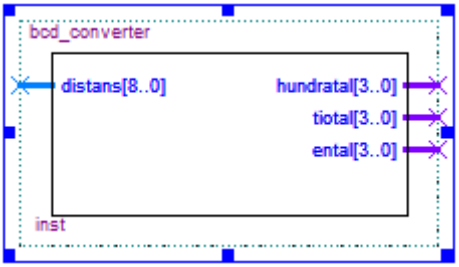
h_counter: Det kommer att räkna från 0 till 799, även om den synliga bilden endast är 640 pixlar bred.
v_counter: Det kommer att räkna från 0 till 525, även om den synliga bilden endast är 480 pixlar hög.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

7.2.7 BCD KONVERTER - KOMPONENT

Komponent namn: ***bcd_converter***
*Instansnamn****: b1v_inst_bcd_converter***

7.2.7.1 SYSTEM ARKITEKTUR

Denna komponent får avståndet i binär för att sen omvandlar i BCD för att kan visa på 7-segment display. Det använder metoden flyttar till vänster och adderar 3(shift-and-add-3.).
VCD CONVERTER-komponentens symbol visas i Figur 13 och tabell 12 visar In- och utsignalerna.



Figur 13. VGA kontroller för ultraljudsensor

Tabell 12. In- och utsignalerna till BCD_KONVERTER filen

Signal	Namn	Riktning	Typ
Avstånd	distans	in	std_logic_vector(8 downto 0)
Bcd om bas 10	undratal	out	std_logic_vector(3 downto 0)
Bcd om bas 10	tiotal	out	std_logic_vector(3 downto 0)
Bcd om bas 10	ental	out	std_logic_vector(3 downto 0)

Ingången signaler
distans: avstånd i centimeter.

Utgången signaler
undratal: bcd utgången som hundratal på bas 10.
tiotal: bcd utgången som tiotal på bas 10.
ental: bcd utgången som ental på bas 10.

7.2.8 SJU_SEG_DISPLAYER - KOMPONENT

Komponent namn: ***sju_displayer_komponent***

Instansnamn: ***b2v_inst_sju_displayer_komponent_1,***
b2v_inst_sju_displayer_komponent_2,
b2v_inst_sju_displayer_komponent_3

7.2.8.1 SYSTEM ARKITEKTUR

Denna komponent styrprogram för 7-segment display för att visar avståndet.
SJU SEGMENT DISPLAYER-komponentens symbol visas i Figur 14 och tabell 13 visar In- och utsignalerna.



Figur 14. VGA kontroller för ultraljudsensor

Tabell 13. In- och utsignalerna till BCD_KONVERTER filen

Signal	Namn	Riktning	Typ
Avstånd på bas 10	tal_in	in	std_logic_vector(3 downto 0)
Styrdata till HEX6	hex_out	out	std_logic_vector(6 downto 0)

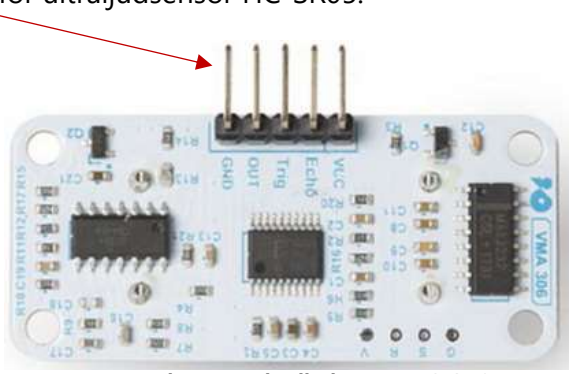
Ingången signale
tal_in: binärt tal (0-15).

Utgången signaler
hex_out: 7-segment display.

7.3 ULTRALJUDSENSOR HC-SR05

För att utveckla projektet det använd avståndsmätaren med ultraljud för Arduino. Når upp till 4,5 m med 3 mm noggrannhet (15° vinkel). Drivspänning: 5 V. Strömförbrukning: 2 mA. Frekvens: 40 kHz. Modell: HC-SR05.

I Figur 14 visar pinar för ultraljudsensor HC-SR05.



Figur 14. Ultraljudsensor HC-SR05

7.3.1 I/O PINNAR

Sensoranslutningen med DE10-lite kort är mycket enkel. Vi kan göra det med en protobord eller direkt med ledningar. För att få sensorn att fungera krävs 4 signaler:

- VCC : får 5V från DE10-lite kort
- GND : GND
- ECHO : Input. När echo är positive flank börjar timer.
- TRIGGER : Output aktivering. 10us digital pulse.
- OUT : Det använder inte på projektet.

Figur 15 visar pinnar för DE10-Lite kortet.

Node Name	Location
clock_50	PIN_N14
echo	PIN_AB17
ental[6]	PIN_C17
ental[5]	PIN_D17
ental[4]	PIN_E16
ental[3]	PIN_C16
ental[2]	PIN_C15
ental[1]	PIN_E15
ental[0]	PIN_C14
hundratal[6]	PIN_B22
hundratal[5]	PIN_C22
hundratal[4]	PIN_B21
hundratal[3]	PIN_A21
hundratal[2]	PIN_B19
hundratal[1]	PIN_A20
hundratal[0]	PIN_B20
reset_n	PIN_F15
tiotal[6]	PIN_B17
tiotal[5]	PIN_A18
tiotal[4]	PIN_A17
tiotal[3]	PIN_B16
tiotal[2]	PIN_E18
tiotal[1]	PIN_D18
tiotal[0]	PIN_C18
trigger	PIN_AA17
vga_b[3]	PIN_N2
vga_b[2]	PIN_P4
vga_b[1]	PIN_T1
vga_b[0]	PIN_P1
vga_g[3]	PIN_R1
vga_g[2]	PIN_R2
vga_g[1]	PIN_T2
vga_g[0]	PIN_W1
vga_hs	PIN_N3
vga_r[3]	PIN_Y1
vga_r[2]	PIN_Y2
vga_r[1]	PIN_V1
vga_r[0]	PIN_AA1
vga_vs	PIN_N1

Figur 15. Pinnar för DE10-Lite kortet

7.4 SDC FILEN

På figur 2 SDC-filen som visar system klockan generering "SYSTEM_CLOCK" från ingående klockan "CLOCK_50". En "ECHO" som en input-klock. En intern klockan "CLK_25" som är en del av "CLOCK_50". Den "CLK_25" det fungerar för att kopplar register inuti cells av systemet. Som sista del av SDC-filen visar ett undantag tid för är tvinga att TimeQuest verktyget ignorerar dessa bestämda vägar.

Figur 16 visar TimeQuest SDC-fil för ”uncertainties”.

```
#SDC file: vhdl2_ingenjorsjobb.sdc

#-- Detta kommando beräknar och tillämpar konfigurationen och behåller klockans
#-- "uncertainties" för varje överföring av clock-to-clock som finns i designen.
#-- Beräkningen av "uncertainties" försenas till nästa kall update_timing_netlist.
derive_clock_uncertainty


#-- skapa en clock "SYSTEM_CLOCK" med en tidperiod av 20ns
create_clock -name SYSTEM_CLOCK -period 20.000 [get_ports {clock_50}]

#-- skapa en clock "ECHO" med en tidperiod av 20ns eftersom TimerQuest
#-- ta den signal som en input-klock
create_clock -name ECHO -period 20.000 [get_ports {echo}]

#-- Genererar en intern klock "CLK_25" från system klockan "clk"
create_generated_clock -name CLK_25 -source [get_ports {clock_50}] \
        -divide_by 2 [get_registers {vhdl2_uppgift_4:biv_inst_vhdl2_uppgift_4|tmp_clk_25mhz}]


#-- Ignorerar tidsbegränsningarna på dessa bestämda vägar.
set_false_path -from [get_ports {reset_n echo}]
set_false_path -to [get_ports {trigger ental[0] ental[1] ental[2] ental[3] ental[4] ental[5] ental[6] tiotal[0] \
                               tiotal[1] tiotal[2] tiotal[3] tiotal[4] tiotal[5] tiotal[6] \
                               hundratal[0] hundratal[1] hundratal[2] hundratal[3] hundratal[4] hundratal[5] hundratal[6] \
                               vga_b[0] vga_b[1] vga_b[2] vga_b[3] \
                               vga_g[0] vga_g[1] vga_g[2] vga_g[3] \
                               vga_r[0] vga_r[1] vga_r[2] vga_r[3] \
                               vga_hs vga_vs}]
```

Figur 16. SDC-fil för ”uncertainties” av Ultraljudsensor

8. SETTINGS OCH ASSIGNMENTS FÖRÄNDRING

8.1 SETTINGS

Det verktygsinställning som används beskrivs i detta kapitel. I ”settings” har inga parametrar förändrats, mer än vilka filer som ska ingå i projektet, vilken testbänk och vilken SDC-fil som ska användas.

8.2 ASSIGNMENTS

Det finns inga ändring på verktygsinställning ”Assignment Editor” för optimerar konstruktionen.

9. VERIFIERING

9.1 TESTBÄNKEN

Syftet av test bench är verifierar att projektet "vhdl2_ingenjorsjobb" att fungerar korrekt. På test bench det finns en "sensor_test" process som simulerar echo ingång signal när är på negativ flank och det finns inte ingen mätare och när den signalen aktiverar sig med positiv flank för att börjar att mäter avståndet.

Figur 17 visar teststimuli som bygger på testprotokollet för programmet.

```
constant sys_clk_period : TIME := 50 ns;
clock_50 <= NOT clock_50 after 10 ns;  --50 MHz -- varje 20 ns byta värde

sensor_test : PROCESS
BEGIN
      reset_n  <= '0';
      wait for 3*sys_clk_period;
      reset_n  <= '1';  -- switchs-reset upp
      wait for 6*sys_clk_period;

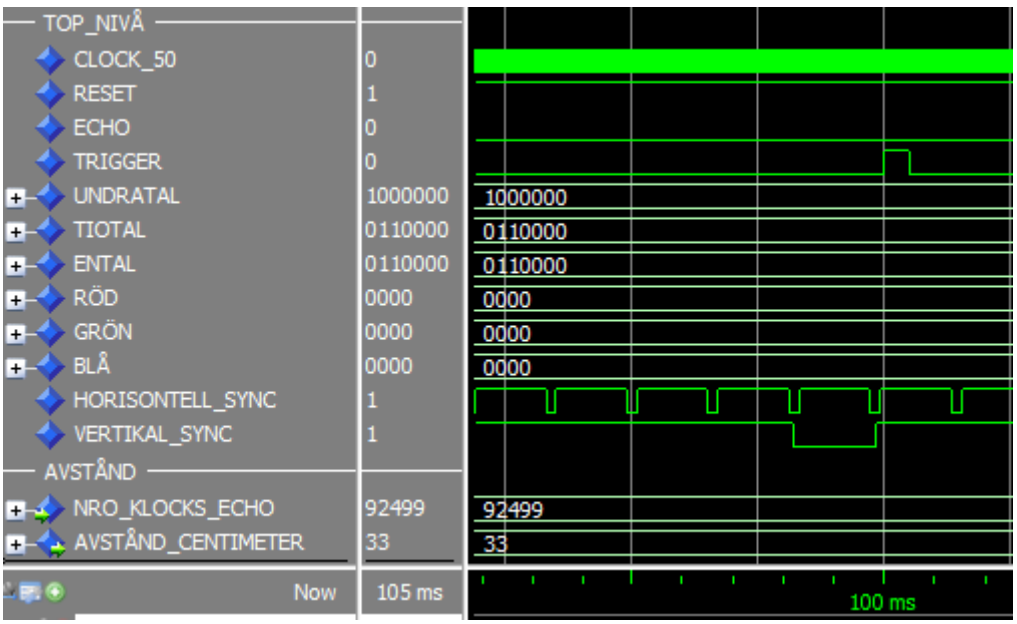
      echo <= '0';
      wait for (sys_clk_period/2)*60000;
      echo <= '1';
      wait for (sys_clk_period/2)*18000; -- Röd (<= 10cm)
      echo <= '0';
      wait for (sys_clk_period/2)*60000;
      echo <= '1';
      wait for (sys_clk_period/2)*42000; -- Grön (> 10cm and <= 20cm)
      echo <= '0';
      wait for (sys_clk_period/2)*60000;
      echo <= '1';
      wait for (sys_clk_period/2)*58000; -- blå (> 20cm and <= 30cm)
      echo <= '0';
      wait for (sys_clk_period/2)*60000;
      echo <= '1';
      wait for (sys_clk_period/2)*74000; -- Vit (> 30cm)
      echo <= '0';
      wait for (sys_clk_period/2)*60000;

WAIT;
END PROCESS sensor_test;
```

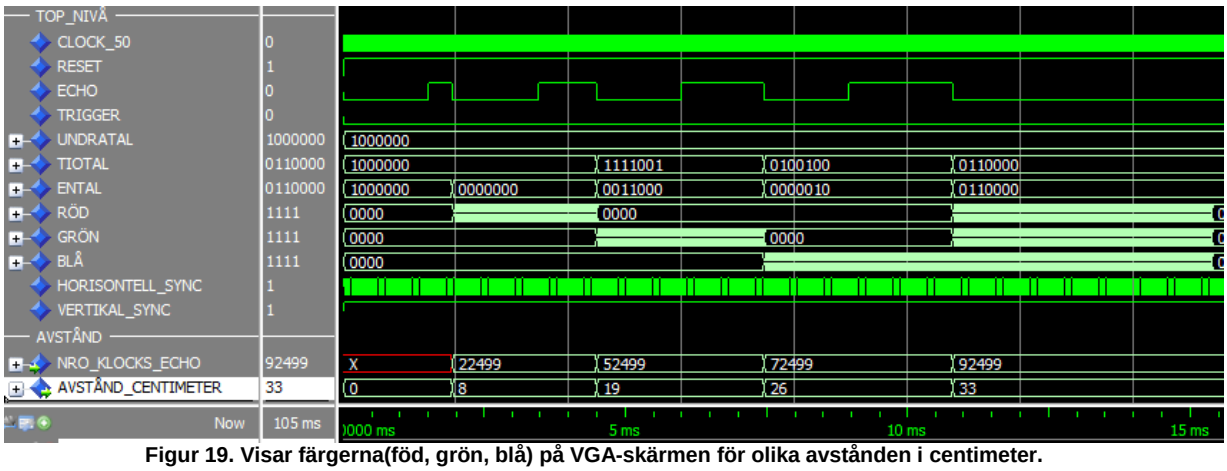
Figur 17. Tesbänken

9.2 RESULTATEN FRÅN SIMULERINGEN

Följande figurer (figur 18-19) visar olika signaler efter verifiering i ModelSim



Figur 18. Efter (100ms+20us) pin TRIGGER av sensoren får en puls av minst 10us från FPGA.



Figur 19. Visar färgerna(föd, grön, blå) på VGA-skärmen för olika avstånden i centimeter.

10. ANALYS

10.1 FELMEDDELANDEN

- Warning (292013): Feature LogicLock is only available with a valid subscription license.

  Man måste köpa licens för att kan ha hela tillgång för denna funktion.

- Warning (15714): Some pins have incomplete I/O assignments. Refer to the I/O Assignment Warnings report for details

  Det är en meddelanden som kan ignoreras eftersom alla pinnar här sina respektive assignments.

- Warning (169177): 3 pins must meet Intel FPGA requirements for 3.3-, 3.0-, and 2.5-V interfaces. For more information, refer to AN 447: Interfacing MAX 10 Devices with 3.3/3.0/2.5-V LVTTL/LVCMOS I/O Systems.

  Denna meddelande är skapat automatiskt i Quartus programvaran och det kan inte att raderar. Denna varning skapas automatiskt när använder ingångar av 2.5-3.3V vccio.

10.2 FREKVENS OCH STORLEK FÖRE OPTIMERING

Denna del visar max frekvens för hela konstruktionen och varje delsystem.

10.2.1 MAX FREKVENS FÖR HELA KONSTRUKTIONEN

Denna konstruktion är kompilerade i en 50 MHZ som systemklocka och har det en klocka av 25 MHZ(CLK_25) för att köras med VGA skärmen. Resurser som används i konstruktionen det passade bra på VGA-kortet men kan det optimerar sig bättre med advisor verktygen.

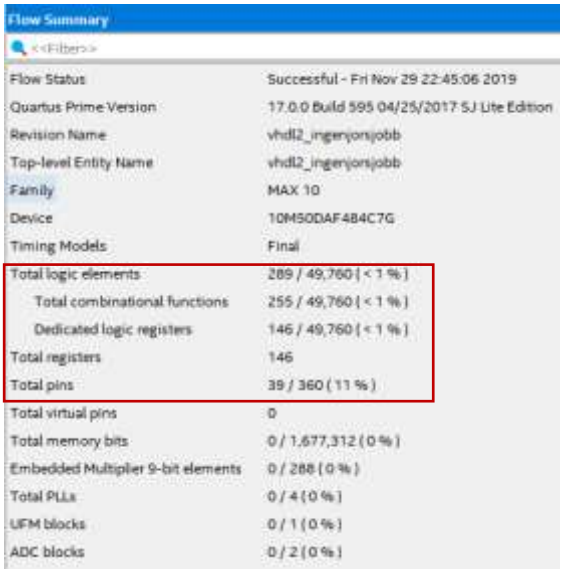
Tabell 14 visar max frekvens för hela konstruktionen.

Tabell 14: Max frekvens av konstruktionen.

Modell	**Fmax systemklocka**
Slow 1200mV 85C Model	139.16 MHz
Slow 1200mV 0C Model	149.5 MHz
Modell	**Fmax CLK_25**
Slow 1200mV 85C Model	259.07 MHz
Slow 1200mV 0C Model	276.85 MHz

10.2.2 STORLEK FÖR HELA KONSTRUKTIONEN OCH DEL SYSTEM

Figur 20 visar konstruktionens resursanvändning, figur 21 visar delsystem resursanvändning.



Flow Summary

Flow Status	Successful - Fri Nov 29 22:45:06 2019
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	vhdl2_ingenjorsjobb
Top-level Entity Name	vhdl2_ingenjorsjobb
Family	MAX 10
Device	10M50DAF484C7G
Timing Models	Final
Total logic elements	289 / 49,760 (< 1 %)
Total combinational functions	255 / 49,760 (< 1 %)
Dedicated logic registers	146 / 49,760 (< 1 %)
Total registers	146
Total pins	39 / 360 (11 %)
Total virtual pins	0
Total memory bits	0 / 1,677,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 288 (0 %)
Total PLLs	0 / 4 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 2 (0 %)

Figur 20: Konstruktionens resursanvändning.

Figur 21 visar resursanvändning för varje delsystem. Nummer som är utanför parentes indikerar antal av resurser som kan använda varje entity och nummer som är utanför parentes indikerar antal av resurser som används varje entity.

Analysis & Synthesis Resource Utilization by Entity

Compilation Hierarchy Node	Combinational ALUTs	Dedicated Logic Registers	Memory Bits	UFM Blocks	DSP Elements	DSP 9x9	DSP 18x18	Pins	Virtual Pins	ADC blocks
|vhdl2_ingenjorsjobb	254 (0)	146 (22)	0	0	0	0	0	39	0	0
|bcd_converter:b1v_inst_bcd_converter|	36 (36)	0 (0)	0	0	0	0	0	0	0	0
|color_delar:b1v_inst_color_delar|	14 (14)	5 (5)	0	0	0	0	0	0	0	0
|distans_raknaren:b1v_inst_distans_raknaren|	36 (36)	24 (24)	0	0	0	0	0	0	0	0
|echo_cont:b1v_inst_echo_cont|	23 (23)	24 (24)	0	0	0	0	0	0	0	0
|sju_seg_displayer:b2v_inst_sju_seg_displayer_1|	9 (9)	0 (0)	0	0	0	0	0	0	0	0
|sju_seg_displayer:b2v_inst_sju_seg_displayer_2|	7 (7)	0 (0)	0	0	0	0	0	0	0	0
|sju_seg_displayer:b2v_inst_sju_seg_displayer_3|	7 (7)	0 (0)	0	0	0	0	0	0	0	0
|trigger_cont:b1v_inst_trigger_cont|	71 (71)	26 (26)	0	0	0	0	0	0	0	0
|vhdl2_uppgift_4:b1v_inst_vhdl2_uppgift_4|	51 (1)	45 (1)	0	0	0	0	0	0	0	0
|counter:b3v_inst_counter|	41 (41)	24 (24)	0	0	0	0	0	0	0	0
|display_rgb:b3v_inst_display_rgb|	9 (9)	20 (20)	0	0	0	0	0	0	0	0

Figur 21: Delsystem resursanvändning.

10.3 EFFEKTFÖRBRUKNING UPPSKATTNING

PowerPlay Power Analyzer Summary

<<Filter>>

PowerPlay Power Analyzer Status	Successful - Tue Dec 03 20:23:21 2019
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	vhdl2_ingenjorsjobb
Top-level Entity Name	vhdl2_ingenjorsjobb
Family	MAX 10
Device	10M50DAF484C7G
Power Models	Final
Total Thermal Power Dissipation	140.37 mW
Core Dynamic Thermal Power Dissipation	3.69 mW
Core Static Thermal Power Dissipation	90.13 mW
I/O Thermal Power Dissipation	46.55 mW
Power Estimation Confidence	High: user provided sufficient toggle rate data

Figur 22: Effektförbrukning med toggle rate på 12,5% och typiska driftförhållanden.

10.4 TIDSVALIDERING

Timingkraven definierad i SDC-filen möttes, inga röda siffror rapporterades i TimeQuest. Den kortaste slacktiden visas i Figur 23.

Slow 1200mV 85C Model

Command Info | Summary of Paths

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	7.170	echo_cont:b1v_inst_echo_cont|sg_count_echo[10]	tmp_echo_count[10]	SYSTEM_CLOCK	ECHO	10.000	-0.936	1.875
2	7.198	echo_cont:b1v_inst_echo_cont|sg_count_echo[9]	tmp_echo_count[9]	SYSTEM_CLOCK	ECHO	10.000	-0.938	1.847
3	7.645	echo_cont:b1v_inst_echo_cont|sg_count_echo[12]	tmp_echo_count[12]	SYSTEM_CLOCK	ECHO	10.000	-0.998	1.340
4	7.664	echo_cont:b1v_inst_echo_cont|sg_count_echo[1]	tmp_echo_count[1]	SYSTEM_CLOCK	ECHO	10.000	-0.938	1.381
5	7.677	tmp_echo_count[1]	distans_rakna..ocks_echo[23]	ECHO	SYSTEM_CLOCK	10.000	0.701	3.007
6	7.677	echo_cont:b1v_inst_echo_cont|sg_count_echo[11]	tmp_echo_count[11]	SYSTEM_CLOCK	ECHO	10.000	-0.928	1.378
7	7.692	tmp_echo_count[0]	distans_rakna..ocks_echo[23]	ECHO	SYSTEM_CLOCK	10.000	0.701	2.992
8	7.707	echo_cont:b1v_inst_echo_cont|sg_count_echo[6]	tmp_echo_count[6]	SYSTEM_CLOCK	ECHO	10.000	-0.938	1.338
9	7.709	echo_cont:b1v_inst_echo_cont|sg_count_echo[0]	tmp_echo_count[0]	SYSTEM_CLOCK	ECHO	10.000	-0.938	1.336
10	7.731	echo_cont:b1v_inst_echo_cont|sg_count_echo[7]	tmp_echo_count[7]	SYSTEM_CLOCK	ECHO	10.000	-0.938	1.314
11	7.731	echo_cont:b1v_inst_echo_cont|sg_count_echo[2]	tmp_echo_count[2]	SYSTEM_CLOCK	ECHO	10.000	-0.938	1.314
12	7.743	echo_cont:b1v_inst_echo_cont|sg_count_echo[4]	tmp_echo_count[4]	SYSTEM_CLOCK	ECHO	10.000	-0.938	1.302
13	7.764	tmp_echo_count[3]	distans_rakna..ocks_echo[22]	ECHO	SYSTEM_CLOCK	10.000	0.701	2.920

Figur 23: Timingrapport från TimeQuest.

11. OPTIMERING KONSTRUKTIONEN

Igenom ”Resource Optimization Advisor” slå på optimering för area i kompilatorn, ange i syntetiseraren att använda minimalt med resurser och aktivera mappning av logik till minne.

Igenom ”Timing Optimization Advisor” kommer att ha en högre maxfrekvens kompilera genom optimering för klockhastighet, att sätta på ”router timing optimization level” som maximum. Alla dessa rekommendationer använder mer resurser för att ha en bättre I/O timing och för att ha en bättre "Hold Time & Minimum Delay Timing".

Igenom ”Power Optimization Advisor”, den konstruktionen kommer att ha en bättre optimering för att at en effektiv effektförbrukning. Den kommer att försöka maximera prestanda, minimera användningen av logik eller balansera hög prestanda med minimal användning av logik. Vissa av dessa tips ger ökad resursanvändning och andra lägre prestanda.

12. VALIDERING KONSTRUKTIONEN

12.1 TESTPROTOKOLL FÖR VALIDERING

Tabell 15 visar testprotokoll för validering och figur 20 visar validering i VGA-skärmen.

Tabell 15. Testprotokoll för validering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Trigger får 10us från FPGA	Trigger puls(100ms+20us)	Ja
2	Echo är positiv flank = ’1’	start counter och räknar nro_klocks_echo	Ja
3	Echo är negativ flank = ’0’	slut counter och räknar nro_klocks_echo total	Ja
4	Avstånd <=10 cm	Visar röd färg	Ja
5	Avstånd >10 cm och <=20	Visar grön färg	Ja
6	Avstånd >20 cm och <=30	Visar blå färg	Ja
7	Avstånd >30	Blandar alla färger för att visa vit farg	Ja

12.2 VALIDERING FILM

https://www.youtube.com/watch?v=Hsf8gtnUw2A&feature=youtu.be

13. REFERENSER

- Sensorn HC-SR04: https://www.geekfactory.mx/tutoriales/tutoriales-arduino/sensor-ultrasonico-hc-sr04-y-arduino/

- BCD convert : https://en.wikipedia.org/wiki/Double_dabble

- https://geekelectronica.com/medicion-de-distancia-con-sensor-ultrasonico-y-arduino/

- Pulsgivarens rotation: https://www.youtube.com/watch?v=UfIZBWesN2E

- Motordrivare rotation: https://www.youtube.com/watch?v=QKP9xu0cdZA